A Bit-Exact, Processor-Free FPGA Adaptive Noise Canceller: Fair Comparison of LMS, NLMS and Divider-Free NLMS-Shift on Cyclone IV E

Satvik Garg & Aditya Gupta & Jaffino G[a,*]

[a]*School of Electronics Engineering (SENSE), Vellore Institute of Technology, Vellore, India*

Abstract

Adaptive noise cancellation with a reference microphone is a classic use of the least-mean-square (LMS) family, and a small field-programmable gate array (FPGA) can run it at audio rates without a processor. This paper presents ULLAP, a 12-bit, 32-tap noise canceller for the DE2-115 board (Cyclone IV E) with a WM8731 codec. One serial core with a single shared multiplier implements LMS, normalised LMS (NLMS) with a restoring divider, and NLMS-Shift, which replaces the division by a leading-one detector and a shift. With step sizes tuned on held-out seeds, NLMS-Shift matches NLMS within the 95% confidence interval on stationary and amplitude-step noise (22.37 and 24.33 dB over 20 seeds) and is about 2 dB better than LMS on amplitude steps, while needing 72 instead of 130 busy cycles per sample. NLMS-Shift is shown to equal NLMS with a level-dependent step between 0.5 and 1 times the nominal one. On real recorded noise and speech the filters remove 11.7–14.9 dB and raise the short-time objective intelligibility from 0.69–0.78 to 0.95–0.98; without speech they remove 18.5–31.9 dB, so the speech, not the noise, limits them. A 12-bit word keeps the quantisation loss below 0.4 dB on broadband noise. On the board, all 724,459 captured output samples of the three algorithms equal the integer model's, and the measured sample rate, latency and busy cycles match the register-transfer-level design.

*Corresponding author. [TBD: e-mail and author order to be confirmed by the authors]

Keywords: adaptive noise cancellation, NLMS, divider-free normalisation, FPGA, fixed-point arithmetic, bit-exact verification

1. Introduction

Unwanted sound is part of almost every working day: the hum of an air conditioner in an office, the compressor next to a production line, the road noise in a car cabin, the babble around a person who wears a hearing aid. When a second sensor can pick up the noise close to its source, the noise that reaches the main microphone can be predicted from that reference and subtracted. An adaptive filter learns the acoustic path between the two sensors while it runs, so it keeps working when the path or the noise changes. The least-mean-square (LMS) algorithm and its normalised form (NLMS) are still the usual choice for this filter, because each sample costs only a few multiply–accumulate operations per tap and the behaviour is well understood [1].

For small embedded devices the filter has to run in real time with a fixed, short delay and very little energy, and here a field-programmable gate array (FPGA) is attractive: the datapath is built for the algorithm, the latency is fixed to the clock cycle, and no processor or operating system is needed. The two algorithms are not equally convenient in hardware, though. LMS needs only multiplications and additions, but its convergence speed depends on the input level. NLMS divides the step size by the input power and so removes this dependence, at the price of one division per sample. A divider is slow and large in logic, and much recent work therefore replaces it by an approximation: an approximate Newton–Raphson iteration [2], canonical-signed-digit reciprocals [3], or a binary reciprocal built from a leading-one detector and a shifter [4]. The simplest member of this family rounds the input power to a power of two, so that the division becomes a plain shift; we call this variant NLMS-Shift.

Reading this literature, we found two things missing. First, the variants are seldom compared with the same tuning effort. A reported gain can then come from a badly tuned baseline rather than from the algorithm. This happened in an earlier version of our own work: a 5.7 dB advantage of NLMS-Shift over NLMS disappeared once NLMS was given a fair regularisation constant. Second, hardware papers report area, clock rate and power, which show that a design fits on the chip, but they rarely show that the chip computes the same samples as the model that produced the accuracy figures.

This paper presents ULLAP, a 12-bit, 32-tap adaptive noise canceller for the low-cost Terasic DE2-115 board (Intel Cyclone IV E) with its WM8731 audio codec, and uses it to answer both points. The main contributions are:

1. A single serial core that implements LMS, NLMS (restoring divider) and NLMS-Shift with one shared multiplier, so the three are compared on identical datapath, word length and filter length, inside a complete processor-free audio system that closes timing at 50 MHz on Cyclone IV E and is released with all scripts and data.
2. A fair comparison: every algorithm's step size and regulariser are chosen on held-out seeds with the same criterion, and results are reported with 20-seed 95% confidence intervals, on synthetic scenarios and on real recorded noise and speech, together with a reproducible word-length study.
3. A bit-exact verification chain from a Python integer model to the RTL and the board wrapper (86 regression tests including codec, freeze and reset behaviour), and on-board captures of real codec samples in which every output sample of all three algorithms equals the model's.
4. Two exact accounts of the results. First, what the shift changes: NLMS-Shift is NLMS with a level-dependent step between 0.5 and 1 times the nominal one, and NLMS run with the mean of that step reproduces it within 0.3 dB on broadband noise; on tonal input the step jumps at power-of-two boundaries of the window power. Second, the real-recording result: with the speech removed, the same filters remove 18.5–31.9 dB of the same real noise, so the lower noise reduction with speech comes from the speech disturbing the adaptation, not from the noise.

We do not claim the shift normaliser itself as new; Section 2 places it among earlier division-free designs.

The rest of the paper is organised as follows. Section 2 reviews the problem and the related work. Section 3 gives the system overview, Section 4 the algorithms and the fixed-point model, and Section 5 the hardware architecture. Sections 6 and 7 describe the verification and the experiments, Section 8 reports the results, and Sections 9 and 10 discuss them and conclude.

2. Literature review

2.1. The problem: noise in daily life

Noise is now treated as an environmental health risk and not only as a nuisance. More than 113 million people in Europe live with transport noise above 55 dB(A), and traffic noise alone is estimated to cost about 1.3 million healthy life-years every year through its effect on the heart and blood vessels [5]. Hahad et al. [6] review the evidence that long exposure to noise is also linked to mental-health disorders and discuss the biological routes proposed for this link. The World Health Organization estimates that about 1.5 billion people live with some degree of hearing loss, a number that may reach 2.5 billion by 2050 [7]. For these listeners, background noise is the main reason why speech stays hard to follow even with a hearing aid. Reducing noise close to the ear or at the microphone is therefore useful in many places: at the roadside, where measured urban traffic noise lies mostly below 2 kHz [8]; in vehicle cabins [9, 10]; in ventilation ducts and air conditioners [11]; and in hearing aids [12, 13, 14]. Many of these devices are small, battery-powered and cheap, so the processing must be both real-time and low in cost and energy.

2.2. Applications of adaptive noise control

Yang et al. [8] measured road traffic noise at five sites in Beijing and built a feedforward LMS prototype that removes close to 30 dB(A) of single- and dual-frequency noise below 2 kHz. In a car, Jiang et al. [9] let the FxLMS step depend on the reference norm, tuned the two parameters of that rule with particle-swarm optimisation, and tested the system on a dSPACE board: it reached up to 12 dB below 100 Hz and about 3 dB over 20–400 Hz in road tests, where plain FxLMS gave between -0.8 and 0.3 dB. Zhu et al. [10] used the built-in loudspeakers of a car to control the first four engine orders, measured an average reduction of 6.2 dB at 52 ear-level microphones, and found that 93% of the listeners in a jury test felt more comfortable with the system switched on. For hearing aids, Yin and Chen [12] weighted the error of a variable-step adaptive feedback canceller and converged 35% faster than a PEM-IMLMS baseline and 60% faster than PEM-NLMS on a real device. Husstedt et al. [13] measured six commercial behind-the-ear devices and found that impulse noise reduction lowers C-weighted peak levels by up to 20 dB, about 9 dB on average. A recent systematic review of speech enhancement for hearing aids [14] concludes that deep networks do best in

complex scenes, while hybrid systems that keep an adaptive filter offer a better trade-off between latency and computation for wearable devices.

2.3. Adaptive algorithms

Lu et al. surveyed a decade of active noise control algorithms in two parts, one for linear systems [1] and one for nonlinear systems [15]; in both, LMS-type updates remain the base on which most newer methods are built. Several groups combine two filters to get fast convergence and low steady-state error at the same time. Félix et al. [16] proposed a modified convex combination of FXLMS filters and report faster convergence and more attenuation than FXLMS and the earlier convex scheme. Zhu et al. [11] combined an adaptive FxLMS filter with a CNN–GRU network through a convex weight to cancel harmonic noise in a bent air-conditioner pipe. When the loudspeaker feeds back into the reference microphone, the loop can become unstable; Li et al. [17] addressed this with a Youla-parameterised feedforward controller and an adaptive IIR update, and verified it on a duct. These methods improve the cancellation, but they add operations, networks or extra filters to every sample, which is the opposite of what a small FPGA design needs.

2.4. Hardware realisations

Thannoon and Hashim [18] implemented a convex combination of a fast retimed RLS filter and a slow LMS filter on an FPGA to remove power-line interference from ECG records of the MIT-BIH database, and report 24.2% higher SNR and 50% faster convergence than conventional structures. Jagadeeswari et al. [19] applied cut-set retiming to delayed NLMS filters for audio on a Spartan-6 and shortened the critical path from 29.96 to 13.39 ns. Karakaya and Aytar [20] built semi-parallel 16-bit NLMS and variable-step LMS filters on an Artix-7 for underwater sonar and raised the SNR from 8.85 dB to 29.36 and 28.57 dB with 2.51% and 2.91% of the LUTs. Park and Park [21] replaced the adaptive filter by fixed filters selected by a small CNN noise classifier and report 2.3 dB more SNR than LMS and 53% less dynamic power in simulation. Abdi et al. [22] built a narrowband controller with a variable-step LMS notch filter on an FPGA to suppress microphonics in a superconducting accelerator cavity. At the arithmetic level, Nagabushanam et al. [23] used distributed arithmetic with offset-binary coding to build a multiplier-less LMS filter, and da Rosa et al. [24] replaced the multipliers of an LMS power-line canceller by adders and shifters, saving up to 4.98 times in power and 30.46% in area.

2.5. Removing the division from NLMS

The works closest to ours deal with the NLMS division. Guidotti et al. [2] used an approximate Newton–Raphson integer divider in an NLMS interference canceller to save power, and Coelho et al. [25] reduced the cost of adaptive filtering with sum-of-powers-of-two arithmetic. Yao and Huang [3] approximated the reciprocal of the input power with canonical-signed-digit encoded divisors in a 16-tap NLMS FIR filter, and Huang and Yao [4] later proposed a binary reciprocal approximation for an NLMS IIR filter. On the same DE2-115 board used here, their design needs 5.07 nJ per sample, against 8.33 nJ with a 15-bit SRT divider, and lowers the energy per sample by 39.1%, 6.5%, 14.4% and 5.6% relative to SRT, Newton–Raphson, CSD and lookup-table methods [4]. They also note that the earlier idea of quantising the divisor to a single power of two, which makes the division a shift, gives a large approximation error. That plain shift is exactly NLMS-Shift; Section 8.4 shows that its error acts only as a level-dependent step factor between 0.5 and 1.

None of the works above compares LMS, a true divider and a shift normaliser on one core with equal tuning and statistical tests, and few test a complete audio system with a codec on a board. ULLAP contributes this equal-effort three-way comparison, the bit-exact chain from the model to the hardware, and an evaluation on real recordings.

3. System overview

Fig. 1 shows the complete system. The noise source v reaches the primary microphone through an unknown acoustic path h and mixes there with the wanted signal s; a reference microphone close to the source picks up v itself. The two microphone signals enter the left and right LINE IN channels of the WM8731 codec, which digitises them and sends them to the FPGA over an I^{2}S link. Inside the FPGA, the I^{2}S receiver delivers one pair $(x[n], d[n])$ per sample period to the ANC core, the core returns the error $e[n]$, which is the cleaned signal, and the I^{2}S transmitter sends it back to the codec DAC and LINE OUT. An I^{2}C engine programs the codec once after reset. Two slide switches select bypass and freeze the adaptation, a push button resets the system, and four test points on the GPIO header expose the sample strobe, the output strobe, the busy flag and the frame clock for timing measurements. The whole design is written in plain Verilog and uses neither a processor nor vendor IP.

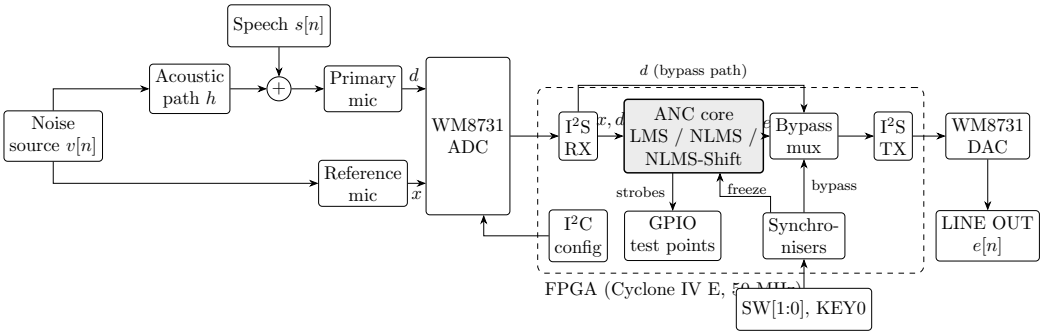


Figure 1: Block diagram of the ULLAP system. The acoustic part on the left is simulated in the experiments (Section 7); everything from the codec on runs on the DE2-115 board.

4. Algorithms and fixed-point model

4.1. Signal model and cost function

The reference sensor observes the noise, and the primary sensor observes the speech plus the noise filtered by the acoustic path and a small sensor noise n_s:

$$x[n] = v[n], \tag{1}$$

$$d[n] = s[n] + \sum_{j=0}^{M-1} h_j \, v[n-j] + n_s[n]. \tag{2}$$

With the reference vector $\mathbf{x}[n] = [x[n], \ldots, x[n-L+1]]^T$ and the weights $\mathbf{w}[n] = [w_0[n], \ldots, w_{L-1}[n]]^T$, the filter estimates the noise part of $d[n]$, and the error is the system output:

$$y[n] = \mathbf{w}^T[n]\,\mathbf{x}[n] = \sum_{i=0}^{L-1} w_i[n]\, x[n-i], \tag{3}$$

$$e[n] = d[n] - y[n]. \tag{4}$$

Because s is uncorrelated with v, minimising the mean-square error

$$J(\mathbf{w}) = E\{e^2[n]\} = E\{d^2[n]\} - 2\,\mathbf{w}^T\mathbf{p} + \mathbf{w}^T\mathbf{R}\,\mathbf{w}, \tag{5}$$

$$\mathbf{R} = E\{\mathbf{x}[n]\,\mathbf{x}^T[n]\}, \qquad \mathbf{p} = E\{d[n]\,\mathbf{x}[n]\}, \tag{6}$$

removes the noise without touching the speech. Its minimum is the Wiener solution $\mathbf{w}_o = \mathbf{R}^{-1}\mathbf{p}$, which for $L \geq M$ and white n_s equals the acoustic path.

4.2. LMS, NLMS and NLMS-Shift

LMS replaces the gradient $\nabla J = -2(\mathbf{p} - \mathbf{R}\mathbf{w})$ by its instantaneous estimate $-2e[n]\mathbf{x}[n]$, and all three algorithms share the resulting update

$$\mathbf{w}[n+1] = \mathbf{w}[n] + \mu[n]\, e[n]\, \mathbf{x}[n]. \tag{7}$$

They differ only in the step $\mu[n]$. Writing the window power as $P[n] = \|\mathbf{x}[n]\|^2$, the steps of LMS, NLMS and NLMS-Shift are

$$\mu_{\mathrm{LMS}}[n] = \mu, \qquad 0 < \mu < \frac{2}{L\,\sigma_x^2}, \tag{8}$$

$$\mu_{\mathrm{NLMS}}[n] = \frac{\tilde{\mu}}{P[n] + \epsilon}, \qquad 0 < \tilde{\mu} < 2, \tag{9}$$

$$\mu_{\mathrm{Shift}}[n] = 2^{-(k[n]+K)}, \tag{10}$$

$$k[n] = \lfloor \log_2(P[n] + \epsilon) \rfloor + 1, \tag{11}$$

where σ_x^2 is the reference variance and $\epsilon > 0$ a small regulariser. The LMS bound depends on the input level, which is why a fixed μ is either slow at low levels or unstable at high levels; the NLMS bound does not. For the power, the hardware uses the exact recursion

$$P[n] = P[n-1] + x^2[n] - x^2[n-L], \tag{12}$$

which costs two squarings per sample instead of L. In NLMS-Shift the exponent $k[n]$ is the bit length of $P[n] + \epsilon$, so it follows from a leading-one detector, and the multiplication by $\mu[n]$ becomes an arithmetic right shift of $e[n]\,x[n-i]$ by $k[n] + K$ places.

Proposition 1. *For every n, the NLMS-Shift step* (10) *satisfies*

$$\frac{2^{-K}}{2\,(P[n] + \epsilon)} \;\le\; \mu_{\mathit{Shift}}[n] \;<\; \frac{2^{-K}}{P[n] + \epsilon}, \tag{13}$$

so NLMS-Shift always lies within a factor of two below NLMS with $\tilde{\mu} = 2^{-K}$ and keeps the NLMS stability bound $0 < \tilde{\mu} < 2$ for $K \ge 0$.

Proof. By (11), $2^{k[n]-1} \le P[n] + \epsilon < 2^{k[n]}$. Taking reciprocals and multiplying by 2^{-K} gives (13), since $\mu_{\mathrm{Shift}}[n] = 2^{-K}\, 2^{-k[n]}$. □

More exactly,

$$\mu[n] = \frac{r[n]\,2^{-K}}{P[n]+\epsilon}, \qquad r[n] = \frac{P[n]+\epsilon}{2^{\lfloor \log_2(P[n]+\epsilon)\rfloor+1}} \in [0.5, 1), \tag{14}$$

so NLMS-Shift is NLMS with the time-varying step $2^{-K} r[n]$ (Section 8.4). If $u = \log_2(P+\epsilon) - \lfloor \log_2(P+\epsilon) \rfloor$ is spread uniformly over $[0, 1)$, which holds when the input level is unknown over several octaves, the mean ratio is

$$E\{r\} = \int_0^1 2^{u-1}\, du = \frac{1}{2\ln 2} \approx 0.72. \tag{15}$$

4.3. Fixed-point model

All signals and weights are signed $Q1.F$ numbers with $W = 12$ bits and $F = W - 1$ fraction bits. With the rounding shift and the saturation

$$\mathrm{rs}(a, m) = \left\lfloor \frac{a + 2^{m-1}}{2^m} \right\rfloor, \tag{16}$$

$$\mathrm{sat}_B(a) = \min\bigl(\max\bigl(a, -2^{B-1}\bigr),\, 2^{B-1} - 1\bigr), \tag{17}$$

the integer datapath computes, for integer samples $x_i = x[n-i] \cdot 2^F$ and weights w_i,

$$a[n] = \mathrm{sat}_{B_a}\Bigl(\sum_{i=0}^{L-1} w_i\, x_i\Bigr), \tag{18}$$

$$B_a = 2W + \lceil \log_2 L \rceil, \tag{19}$$

$$y[n] = \mathrm{sat}_W\bigl(\mathrm{rs}(a[n], F)\bigr), \tag{20}$$

$$e[n] = \mathrm{sat}_W\bigl(d[n] - y[n]\bigr), \tag{21}$$

$$w_i \leftarrow \mathrm{sat}_W\bigl(w_i + \Delta w_i\bigr), \tag{22}$$

where the weight increments of LMS, NLMS and NLMS-Shift are

$$\Delta w_i^{\mathrm{LMS}} = \mathrm{rs}\bigl((M_\mu\, e[n])\, x_i,\ F + 10\bigr), \tag{23}$$

$$\Delta w_i^{\mathrm{NLMS}} = \mathrm{rs}\bigl(g[n]\, x_i,\ F\bigr), \tag{24}$$

$$g[n] = \mathrm{sat}_{W+4}\left(\left\lfloor \frac{2N[n] + Q[n]}{2Q[n]} \right\rfloor\right), \tag{25}$$

$$\Delta w_i^{\mathrm{Shift}} = \mathrm{rs}\bigl(e[n]\, x_i,\ b(Q[n]) - F + K\bigr), \tag{26}$$

with $\mu = M_\mu\, 2^{-10}$, $N[n] = M_\mu\, e[n]\, 2^{2F-10}$, $Q[n] = P[n] + 2^{2F-E}$ for $\epsilon = 2^{-E}$, and $b(\cdot)$ the bit length of a positive integer. Every narrowing point saturates and sets a sticky flag, so an overflow is never silent. The integer model (`py/fixedpoint.py`, `py/rtl_model.py`) mirrors these widths exactly and generates the golden vectors.

5. Hardware architecture

5.1. Core

Fig. 2 shows the datapath of the core (`rtl/anc_core.v`, vendor-neutral Verilog). It accepts one sample pair per `in_valid` strobe and computes everything with a single multiplier, pipelined in three stages (operand registers, product register, consume stage). A finite-state machine steps through the phases of a sample: two cycles to square the new and the dropped reference sample for (12) (NLMS and NLMS-Shift only), L cycles for the filter sum (18), one cycle for y and e (21), the step computation, and L cycles of weight updates (22). NLMS adds a sequential restoring divider for the gain $g[n]$; NLMS-Shift replaces it by a leading-one detector and a shift-amount register. A `freeze` input holds the weights while filtering continues.

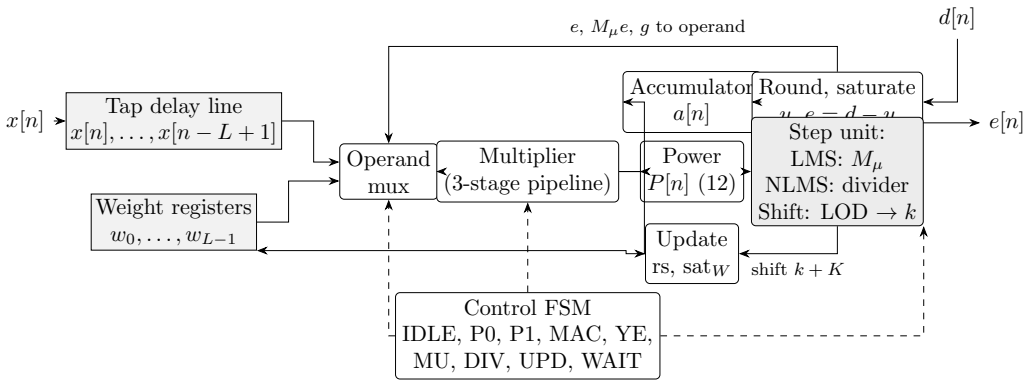


Figure 2: Datapath of the ANC core. One multiplier is shared by the power update, the filter sum, the step computation and the weight updates; only the step unit differs between the three algorithms. Dashed lines are control signals.

The schedule fixes the input-to-output latency and the number of busy

cycles per sample:

$$N_{\mathrm{lat}} = \begin{cases} L+3 & \text{LMS,} \\ L+5 & \text{NLMS, NLMS-Shift,} \end{cases} \tag{27}$$

$$N_{\mathrm{busy}} = \begin{cases} 2L+9 & \text{LMS,} \\ 2L+13+N_{\mathrm{div}} & \text{NLMS,} \\ 2L+8 & \text{NLMS-Shift,} \end{cases} \tag{28}$$

where the restoring divider runs one iteration per dividend bit, $N_{\mathrm{div}} = W + B_\mu + 2F + 4 + \lceil \log_2 L \rceil$, and B_μ is the signed width of M_μ, so the NLMS cycle count grows with the step-size constant. At $L = 32$ and the tuned settings this gives latencies of 35, 37 and 37 cycles and busy times of 73, 130 ($N_{\mathrm{div}} = 53$) and 72 cycles, the values measured on the board (Section 8.6).

5.2. Board wrapper

The DE2-115 wrapper contains an I^{2}C engine that configures the WM8731 at reset (11 registers, acknowledge-checked), an I^{2}S receiver and transmitter clocked by the 50 MHz system clock with the codec's bit and word clocks sampled as data, two-flop synchronisers on the reset and switches, a frame-latched bypass multiplexer, an overrun flag, and four registered test points on the GPIO header. The codec runs as master from a 12.5 MHz clock with the sample-rate register set to 1536 fs, so

$$f_s = \frac{f_{\mathrm{MCLK}}}{1536} = \frac{12.5\ \mathrm{MHz}}{1536} = 8.138\ \mathrm{kHz}, \tag{29}$$

$$N_s = \frac{f_{\mathrm{clk}}}{f_s} = \frac{50\ \mathrm{MHz}}{8.138\ \mathrm{kHz}} = 6144, \tag{30}$$

the rate of the simulation study, with N_s clock cycles per sample. The core is busy for the fraction

$$U = \frac{N_{\mathrm{busy}}}{N_s} \tag{31}$$

of each sample period, which by (28) is 1.19%, 2.12% and 1.17% for LMS, NLMS and NLMS-Shift. No processor or vendor IP is used.

6. Verification

Table 1 summarises the verification. The core testbench compares every $y[n]$ and $e[n]$, the final weights, the sticky flags and the latency with golden

vectors for 68 configurations (W from 8 to 31, L from 2 to 32, saturation corner cases, freeze patterns). The top-level testbench uses a behavioural WM8731 model that checks every I^2C write and follows the programmed sample rate, and runs tests T1–T10: configuration, NACK handling, bypass, end-to-end cancellation against golden vectors, freeze, reset, flags, clocks, a 45-point skew sweep, the 8 kHz rate and the GPIO timing. Mutation tests confirm the testbenches fail on injected faults. Verilator `-Wall` lint leaves no unwaived warning.

Table 1: Verification summary. RTL suites pass in both Icarus Verilog and Questa (`results/metrics/iverilog_regression_summary.txt`, `results/local_run_da0b6f4/`).

Suite	Tests	Result
Core golden vectors (`tb_anc_core`)	68	all pass, bit-exact
Protocol corner cases	4	pass
Freeze hold/toggle variants	3	pass
Top level with codec model (T1–T10)	11	pass
Python unit tests	53	pass
Verilator lint (`-Wall`)	–	0 unwaived
On-board Signal Tap captures (T9)	25	all bit-exact
On-board freeze, DAC word, 10-min soak	4	pass

7. Experimental set-up

Synthetic scenarios follow the reference model $x = v$, $d = s + h * v + n_s$ at 8 kHz with 16,000 samples: stationary coloured noise, amplitude steps ($\times 3$ then $\times 0.25$), an abrupt path switch at the midpoint, and a recorded chord as the noise source. Step sizes and regularisers were selected on seeds 101–103 by maximising the mean final noise reduction over stationary, amplitude-step and path-switch scenarios with zero overflow: LMS $\mu = 0.25$; NLMS $\mu = 0.25$, $\epsilon = 1$; NLMS-Shift $K = 3$, $\epsilon = 0.25$. Results are reported on seeds 1–20. The final noise reduction is measured over the last quarter $\mathcal{N}$ of the samples,

$$\mathrm{NR} = 10 \log_{10} \frac{\sum_{n \in \mathcal{N}} \big(d[n] - s[n]\big)^2}{\sum_{n \in \mathcal{N}} \big(e[n] - s[n]\big)^2} \ \mathrm{dB}, \tag{32}$$

and each table entry is the mean over the $S = 20$ seeds with its 95% confidence interval,

$$\overline{\mathrm{NR}} \pm t_{0.975,\,S-1}\,\frac{\sigma_{\mathrm{NR}}}{\sqrt{S}}, \qquad t_{0.975,19} = 2.093. \tag{33}$$

Two algorithms A and B are compared on the paired per-seed differences $\Delta_k = \mathrm{NR}_{\mathrm{A},k} - \mathrm{NR}_{\mathrm{B},k}$. They are called equivalent when both one-sided tests (TOST) reject, at $p < 0.05$, that the mean difference lies outside the margin $\pm\delta$ with $\delta = 0.5$ dB:

$$H_{01}:\ \bar{\Delta} \le -\delta, \qquad H_{02}:\ \bar{\Delta} \ge \delta. \tag{34}$$

Use of generative AI. [TBD: Authors: describe here, as CSSP requires in the Methods, which AI tools were used for code, analysis, literature search and writing, and state that the authors reviewed and take responsibility for the content.]

For real recordings, four noise files (air conditioner, vacuum cleaner, copy machine, babble) and ten utterances from the public MS-SNSD corpus[1] were resampled to 8 kHz; each seed draws a noise segment, an utterance and a random 16-tap acoustic path, at 0 dB input SNR, with and without speech leakage into the reference at −20 dB. Noise reduction, the short-time objective intelligibility (STOI) measure and the narrow-band perceptual evaluation of speech quality (PESQ, ITU-T Recommendation P.862) are computed after a 2 s convergence window.

8. Results

8.1. Synthetic scenarios

Table 2 shows that NLMS-Shift ties NLMS on stationary noise and amplitude steps and ties LMS on stationary noise and the path switch. Both normalised algorithms are about 2 dB better than LMS on amplitude steps, where LMS's fixed step is either too slow after the drop or too noisy before it: no LMS step size reaches 22 dB on stationary noise and 24 dB on amplitude steps at once, while both NLMS variants do (Fig. 3). On the recorded chord NLMS-Shift is 0.2–0.3 dB worse. Paired per-seed tests (Table 3) make "tie"

[1] `https://github.com/microsoft/MS-SNSD`

Table 2: Final noise reduction, dB, mean [95% CI] over seeds 1–20, $W = 12$, $L = 32$ (`results/paper_numbers.csv`).

Scenario	LMS	NLMS	NLMS-Shift
Stationary	22.39 [22.12, 22.66]	22.28 [22.01, 22.54]	22.37 [22.11, 22.63]
Amplitude steps	22.36 [21.72, 23.00]	24.16 [23.58, 24.73]	24.33 [23.88, 24.78]
Path switch	18.95 [18.81, 19.09]	18.53 [18.39, 18.67]	18.91 [18.76, 19.06]
Recorded chord	5.69 [5.66, 5.73]	5.73 [5.69, 5.76]	5.46 [5.42, 5.50]

precise: every NLMS-Shift versus NLMS difference is inside a ±0.5 dB equivalence margin (TOST, $p < 0.05$), although some are statistically non-zero, for example +0.38 dB on the path switch; the 1.8–2.0 dB advantage of the normalised algorithms over LMS on amplitude steps is significant ($p < 10^{-7}$).

Table 3: Paired differences in final NR over seeds 1–20 (`results/metrics/paired_stats.csv`).

Scenario	Pair	Mean diff. (dB)	95% CI	p (paired t)	TOST ±0.5 dB
Stationary	NLMS-Shift − NLMS	+0.10	[+0.01, +0.18]	0.028	equivalent
Stationary	NLMS-Shift − LMS	-0.02	[-0.07, +0.04]	0.58	equivalent
Stationary	NLMS − LMS	-0.11	[-0.18, -0.04]	0.0038	equivalent
Amplitude steps	NLMS-Shift − NLMS	+0.17	[-0.12, +0.47]	0.23	equivalent
Amplitude steps	NLMS-Shift − LMS	+1.97	[+1.51, +2.43]	2.8e-08	not equiv.
Amplitude steps	NLMS − LMS	+1.79	[+1.38, +2.21]	2.5e-08	not equiv.
Path switch	NLMS-Shift − NLMS	+0.38	[+0.30, +0.47]	1.9e-08	equivalent
Path switch	NLMS-Shift − LMS	-0.04	[-0.07, -0.01]	0.018	equivalent
Path switch	NLMS − LMS	-0.42	[-0.50, -0.34]	1.1e-09	equivalent
Recorded chord	NLMS-Shift − NLMS	-0.27	[-0.30, -0.24]	7.7e-14	equivalent
Recorded chord	NLMS-Shift − LMS	-0.24	[-0.26, -0.22]	2.6e-16	equivalent
Recorded chord	NLMS − LMS	+0.03	[+0.00, +0.06]	0.036	equivalent

Figures 4–6 show single runs (seed 1) at the fair-tuned settings. Fig. 4 gives the running noise reduction in each scenario, Fig. 5 the residual error power against the input noise power for stationary noise, and Fig. 6 the response to the amplitude steps: after the drop at 1.2 s LMS settles about 3 dB below both normalised algorithms, consistent with Table 2.

8.2. Real noise and speech

Table 4 shows that on all four real noises the filter roughly doubles PESQ and brings STOI close to 1, and that the order of the algorithms is the same everywhere: NLMS 0.4–0.6 dB ahead of LMS, NLMS-Shift 0.15–0.35 dB behind NLMS. The noise reduction is lower than in Table 2, but not because the real noise is harder to predict. With the

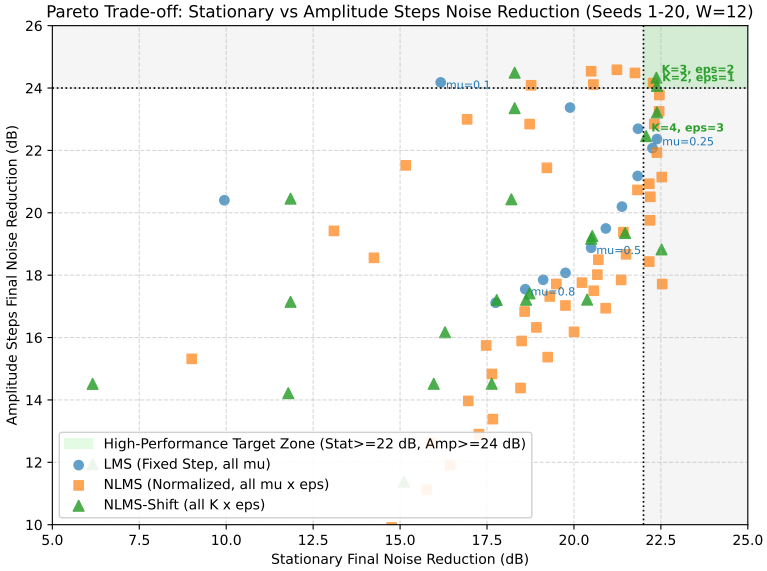


Figure 3: Stationary versus amplitude-step noise reduction across step sizes (seeds 1–20).

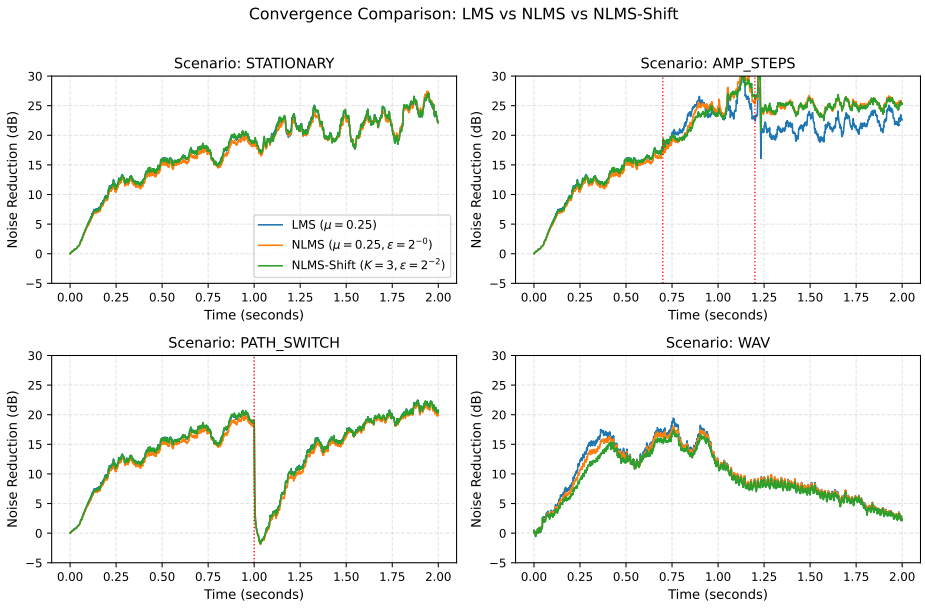


Figure 4: Running noise reduction in the four synthetic scenarios (seed 1, $W = 12$, $L = 32$, fair-tuned settings). Dotted lines mark the amplitude steps and the path switch.

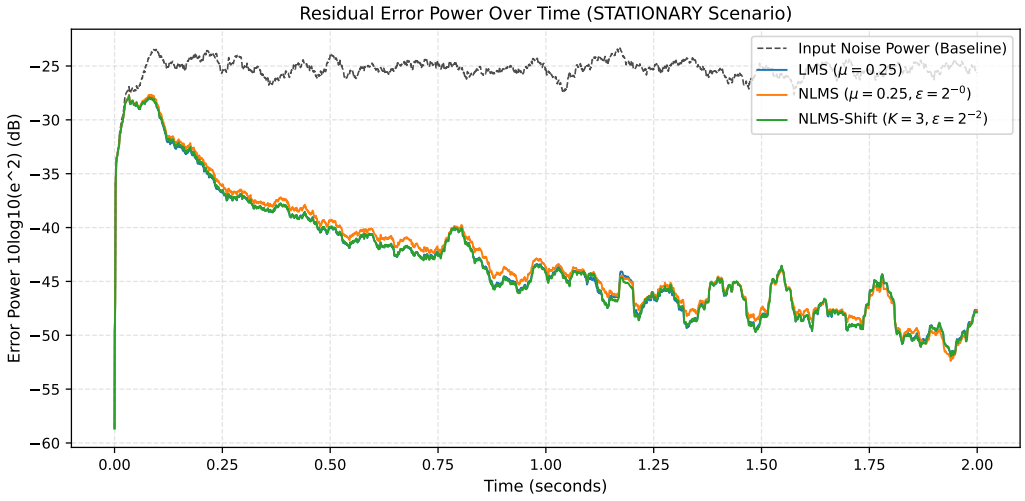


Figure 5: Residual error power over time against the input noise power, stationary scenario (seed 1).

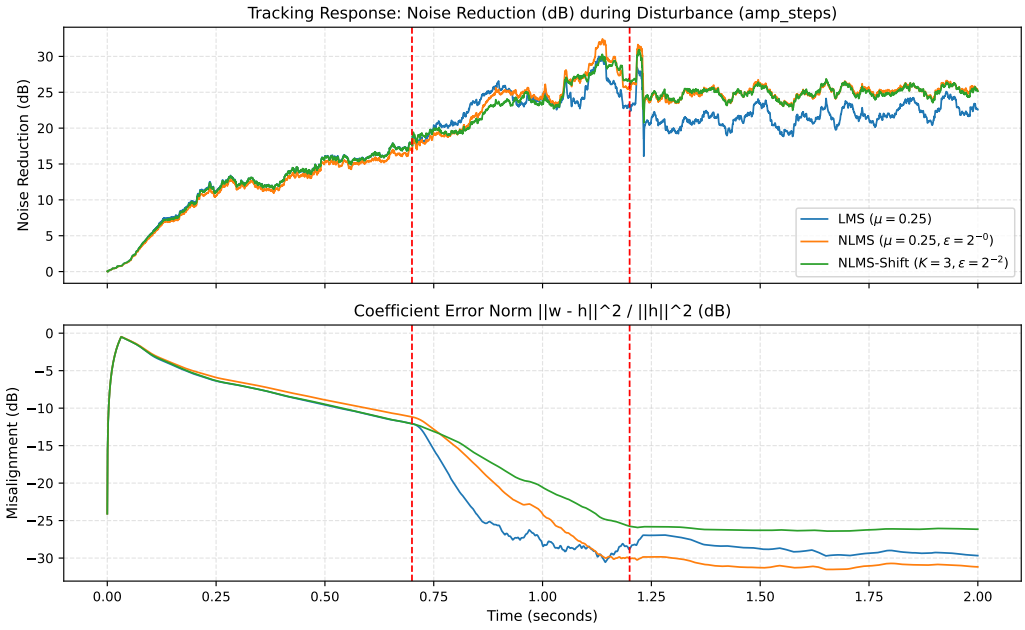


Figure 6: Tracking of the amplitude-step disturbance (seed 1): noise reduction and coefficient misalignment.

Table 4: Real recorded noise and speech (MS-SNSD), fixed point $W = 12$, mean ± 95% CI over 20 seeds, no speech leakage into the reference; STOI and PESQ of e against s, with the unprocessed input in brackets (`results/metrics/real_audio_summary.csv`).

Noise	Metric	LMS	NLMS	NLMS-Shift
Air conditioner	NR (dB)	11.65 ± 0.75	12.04 ± 0.82	11.89 ± 0.80
	STOI (input 0.784)	0.948	0.947	0.948
	PESQ (input 1.54)	2.34	2.30	2.33
Vacuum cleaner	NR (dB)	14.32 ± 0.32	14.90 ± 0.33	14.56 ± 0.35
	STOI (input 0.730)	0.982	0.983	0.982
	PESQ (input 1.39)	2.81	2.81	2.81
Copy machine	NR (dB)	14.02 ± 0.38	14.63 ± 0.34	14.39 ± 0.32
	STOI (input 0.694)	0.977	0.978	0.977
	PESQ (input 1.36)	2.74	2.75	2.73
Babble	NR (dB)	11.95 ± 0.42	12.43 ± 0.49	12.27 ± 0.46
	STOI (input 0.719)	0.956	0.955	0.955
	PESQ (input 1.48)	2.48	2.47	2.47

speech removed from the primary input, the same filters on the same noise segments and paths reach 18.5–19.2 dB on the air conditioner and 22.4–31.9 dB on the other three noises, as much as or more than on synthetic stationary noise (`py/exp_real_audio_nospeech.py`, `results/metrics/real_audio_nospeech.csv`). The speech, uncorrelated with the reference, enters the error $e[n]$ that drives the weight update and costs 6.4–17.9 dB. LMS is ahead of both normalised variants on all four noises without speech and loses most with it (7.6–17.9 dB against 6.4–14.8 dB for NLMS), which reverses the order. Slowing or stopping the adaptation while speech is present is the known remedy; the core's `freeze` input (Section 5.1) could be driven by a voice-activity detector, which is not evaluated here. When −20 dB of the speech leaks into the reference, the filter also partly cancels speech: noise reduction drops by 2.3–4.6 dB, PESQ by 0.25–0.6, and saturation events appear (18,500 over 19.2 million processed samples, most for the air conditioner), so reference-microphone placement matters more than the choice of algorithm (`paper/tab_real_audio_xt.tex`).

8.3. Word length

At the tuned settings, the worst-scenario loss relative to the floating-point run of the same algorithm is 4.1–5.9 dB at $W = 10$, 0.9–1.2 dB at $W = 11$

and 0.3–0.4 dB at $W = 12$ (Fig. 7); 11 bits is the shortest word within the 3 dB criterion and 12 bits was kept for margin. On the tonal chord recording the fixed-point filter outperforms floating point by up to 3.4 dB at $W = 11$ and loses 1.3 dB at $W = 13$: on this narrow-band signal quantisation changes the effective adaptation, so the chord result is sensitive to the word length and should not decide it.

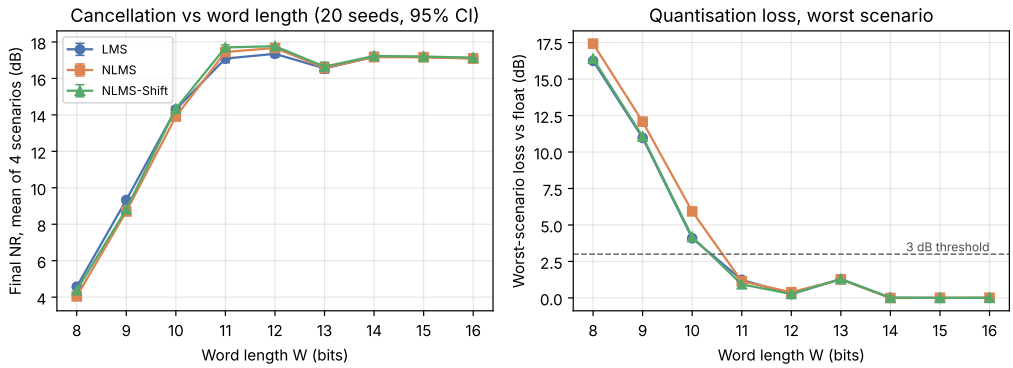


Figure 7: Noise reduction and worst-scenario quantisation loss versus word length (20 seeds).

8.4. Equivalent step size of NLMS-Shift

By (14), the only difference between NLMS-Shift and NLMS is the factor $r[n]$. Averaged over input levels spread uniformly on a logarithmic scale, $E[r] = \int_0^1 2^{u-1}\,du = 1/(2\ln 2) \approx 0.72$. At a given level, however, r is set by where $P + \epsilon$ lies within its octave: it grows with the level and halves when $P + \epsilon$ crosses a power of two. With $L = 32$ and $\epsilon = 2^{-2}$, the crossing $P+\epsilon = 1/2$ lies at a reference RMS of -21.1 dBFS, 3 dB above the simulation level. Fig. 8 sweeps the input level over 12 dB in 0.75 dB steps in the integer model (`py/exp_equiv_step.py`), on the stationary scenario (seeds 1–5) and on a two-tone reference. On broadband noise the window power varies enough to smooth the sawtooth, and the mean ratio $\bar{r}$ moves gradually between 0.56 and 0.78. On the tonal reference P is almost constant, and $\bar{r}$ falls from 0.87 to 0.62 within 0.75 dB of level. In both cases NLMS run with the constant step $\bar{r}\,2^{-K}$ reproduces NLMS-Shift: the final noise reduction agrees within 0.27 dB at all 17 broadband levels and within 0.55 dB at all tonal levels but the one that straddles the crossing (1.7 dB), and the convergence time within 8%. NLMS with the nominal step 2^{-K} differs by up to 3.5 dB and 48%.

So the shift changes nothing in the adaptation but the step size: it turns the fixed NLMS step into a step between 0.5 and 1 times the nominal one that depends on the input level. On broadband input this is a smooth gain change that can be absorbed by the choice of K (and ϵ) for the expected level; on a tonal input the step changes by up to a factor of about 0.7 when the level crosses a power-of-two boundary of $P+\epsilon$, which a fixed-step NLMS does not do.

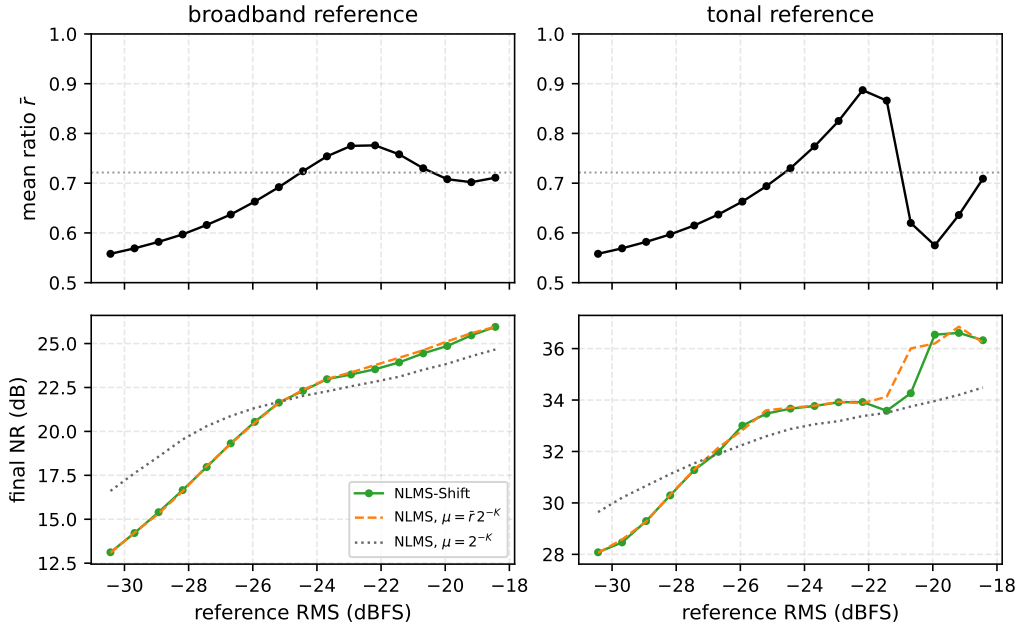


Figure 8: Equivalent step of NLMS-Shift versus reference level (integer model, $W = 12$, $L = 32$, $K = 3$, $\epsilon = 2^{-2}$). Top: mean ratio $\bar{r}$ of (14); dotted: $1/(2\ln 2)$. Bottom: final noise reduction of NLMS-Shift, of NLMS with step $\bar{r}\,2^{-K}$ and of NLMS with step 2^{-K}. Left: stationary scenario, seeds 1–5; right: two-tone reference (`results/metrics/equiv_step.csv`).

8.5. Hardware cost

LMS is about 31% smaller than either normalised variant. NLMS-Shift and NLMS use about the same logic (2,610 and 2,626 logic elements), because the divider's area is replaced by the leading-one detector, shifter and wider update path, but NLMS-Shift has 15% fewer registers, needs 45% fewer

Table 5: Complete board design (core, codec interface, synchronisers, status logic) on Cyclone IV E EP4CE115F29C7, $W = 12$, $L = 32$, Quartus Prime Lite 25.1, commit da0b6f4 (`results/quartus_reports/board_da0b6f4/`, `results/metrics/power_activity.csv`).

	LMS	NLMS	NLMS-Shift
Logic elements	1,792	2,626	2,610
Registers	1,123	1,404	1,186
9-bit multipliers	4	4	4
$F_{\max}$, slow 85 °C (MHz)	70.08	59.75	58.66
Setup slack at 50 MHz, slow 85 °C (ns)	5.731	3.263	2.953
Latency (cycles)	35	37	37
Busy cycles per sample (measured on the board)	73	130	72
RTL bit toggles per sample	5,668	7,146	6,012
Core dynamic power, vectorless (mW)	20.79	24.36	23.77

busy cycles per sample and toggles 16% fewer bits per sample. All three designs meet the 50 MHz constraint with at least +2.953 ns setup slack and +0.401 ns hold slack, with no critical warnings. At 8.138 kHz the busiest variant uses 2% of the 6,144-cycle sample period, so a single core could serve many channels or much longer filters.

The power figures are Quartus vectorless estimates with the default 12.5% toggle rate, which Quartus itself rates as low confidence. Core static power (about 98.7 mW) dominates the 169 to 173 mW total. The dynamic estimates rank the designs LMS < NLMS-Shift < NLMS, as the toggle counts do, but a fixed toggle rate mostly tracks area and ignores that the datapath is idle for 98% of each sample period, so these numbers are upper bounds and not energy per sample. A gate-level, activity-annotated estimate is left for future work: the post-fit netlist flattens the hierarchy that the top-level testbench observes, so it needs a netlist-only testbench. For comparison with [4], which uses the same board, energy per sample should be reported as dynamic power divided by the sample rate from an activity-annotated Power Analyzer run, which this work does not yet have, so no energy comparison with them is claimed. Their design is fully parallel and run at megasamples per second, while ULLAP shares one multiplier and runs at 8.138 kHz, so area and energy are compared per tap and per sample, not as raw totals.

8.6. On-board results

The board evidence is recorded by the FPGA itself with Signal Tap, so it does not depend on an external recorder or oscilloscope. One logic-analyser instance stores, once per sample, the two codec samples entering the core (x and d, register outputs `x_reg` and `d_reg`) and the core output e, with a transitional storage qualifier on `out_valid`, and triggers when the reset button is released; it holds 32,768 samples (4.0 s). A second instance samples the four GPIO test points at 50 MHz. The stimuli are the noise-only versions of the four scenarios, played from a laptop into LINE IN after a 1 kHz calibration tone; the reset is pressed during the tone, so each capture starts from zero weights on real codec data at about -25 dBFS, and no sample in any capture reaches full scale. The analysis script and the raw exports are in the repository (`py/analyze_board_capture.py`, `board_evidence/`).

Bit-exactness. For each of the 12 captures (three algorithms, four scenarios) the integer model of Section 4, started from zero state at the first sample after the reset and fed the captured x and d, reproduces every captured $e[n]$: 351,723 samples in total, 28,672 to 32,768 (3.5 to 4.0 s) per capture, with no mismatch. The fixed-point arithmetic of the silicon is therefore exactly the one evaluated in simulation, also on analogue input with the codec's noise and offsets.

Timing. The GPIO capture gives an `in_valid` period of 6,144 clock cycles (8,138.02 Hz), an `in_valid` to `out_valid` spacing of 38, 36 and 38 cycles for NLMS-Shift, LMS and NLMS (the core latency plus one accept cycle, 0.76, 0.72 and 0.76 μs) and busy times of 72, 73 and 130 cycles, all equal to the RTL simulation of the same parameters. The Signal Tap builds still meet the 50 MHz constraint (setup slack +2.65, +5.68 and +3.78 ns); Table 5 is for the builds without the logic analyser.

Fig. 10 shows the three algorithms on the same amplitude-step stimulus, aligned to the start of the scenario (KEY0 was pressed by hand during the calibration tone, so the reset falls at a slightly different point of the stimulus in each capture). All three settle to about 15 dB in the first 1.5 s and track the step at 2.1 s, where the level of d rises by 7.4 dB, with only a short burst in e: the residual power after the step (-47 to -52 dBFS) is below its level before it (-44 dBFS), so the noise reduction rises to 25 to 30 dB. Over the last quarter of the window the reduction is about 40 dB below 1.5 kHz and 11 to 15 dB near the band edge (3.5 to 3.9 kHz).

Noise reduction on the simulation signal. In a second session the stimulus was the 2 s, seed-1 noise-only signal of each scenario, i.e. exactly the 16,000-

sample signal of the simulations, played once after the calibration tone so that the whole scenario lies inside the capture. Its start is found by correlating the captured x with the stimulus (correlation 0.95 to 0.99), and $\mathrm{NR}_{\mathrm{final}}$ is taken over the same last 25 % as in simulation. All 12 captures (28,672 samples after the reset each) are again bit-exact. Table 6 compares the board with the integer model on the same signal. The board is 2.9 to 10.6 dB below the simulation, by similar amounts for all three algorithms, and the cause is the input level, not the hardware: the codec delivered x and d at about -27 dBFS, 3 dB below the simulation level, and at the tuned step sizes a lower input power slows adaptation within the 2 s. Run at the level the board received, the model reproduces the board within 0.2 dB on stationary and path-switch noise (Fig. 9). For amplitude steps and the recorded noise, whose final quarter is quiet (d at about -41 and -57 dBFS), the board stays 1.7 to 8.1 dB below the level-matched model; there the analogue noise of the playback and codec path (-70 dBFS measured in the silence after the stimulus) and an ADC offset that decays during the first second after the reset are no longer small against d, and for amplitude steps the best 32-tap least-squares filter fitted to the board samples of that window reaches 24 dB instead of 28 dB on the ideal signal. On the board the three algorithms are within 0.7 dB of one another in every scenario.

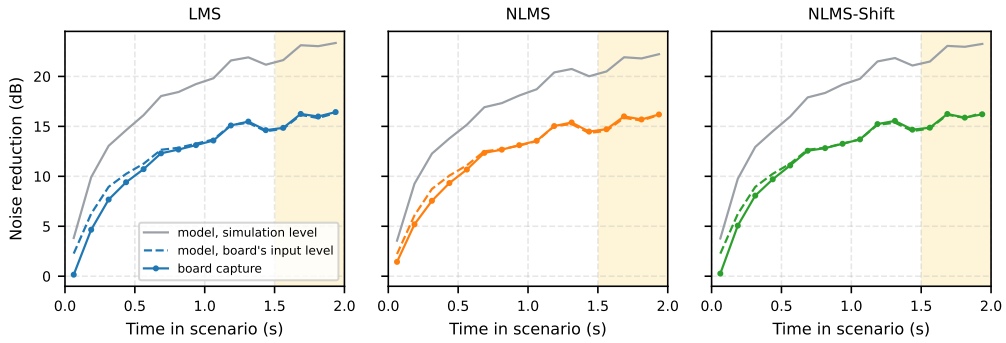


Figure 9: Running noise reduction (0.125 s windows) on the 2 s stationary stimulus: board capture, integer model at the simulation level, and integer model with the stimulus scaled to the level the board received. Shaded: the $\mathrm{NR}_{\mathrm{final}}$ window.

Freeze, DAC word and endurance. With NLMS-Shift on stationary noise, SW[1] was flipped by hand 1.44 s into the scenario. The captured output equals the model with adaptation stopped from that sample on (28,672 sam-

Table 6: On-board NR_{final} (dB) on the 2 s seed-1 stimulus against the integer model on the same signal, at the simulation level and scaled to the level the board received (`results/board/session2/`). Every board capture is bit-exact.

Algorithm	Scenario	Board	Model, sim. level	Model, board level
LMS	stationary	15.88	22.77	15.76
	amplitude steps	17.13	27.71	25.23
	path switch	14.67	19.84	14.47
	recorded noise	5.99	9.11	7.78
NLMS	stationary	15.65	21.60	15.53
	amplitude steps	16.76	27.19	23.64
	path switch	14.03	19.06	14.04
	recorded noise	5.80	8.72	7.45
NLMS-Shift	stationary	15.80	22.68	15.78
	amplitude steps	16.57	25.79	22.10
	path switch	14.40	19.86	14.44
	recorded noise	5.46	8.63	7.32

ples, no mismatch), and the noise reduction stays at 15.2 dB from the switch to the end of the scenario, against 15.4 dB just before it and 15.9 dB had adaptation continued. A third Signal Tap instance recorded the word sent to the DAC: in ANC mode it equals $\{e, 0000\}$ and in bypass (SW[0] = 1) its upper 12 bits equal d, in all 1,792 samples of each capture. Since the LINE OUT signal could not be recorded with the available equipment, this closes the chain up to the DAC interface. After 10 minutes of continuous operation on looped noise, the GPIO timing was unchanged (6,144 cycles per sample, 38 cycles from `in_valid` to `out_valid`, 72 busy cycles) and no saturation, NACK or overrun flag was lit; the build with all three logic-analyser instances still met timing (slack +2.93 ns).

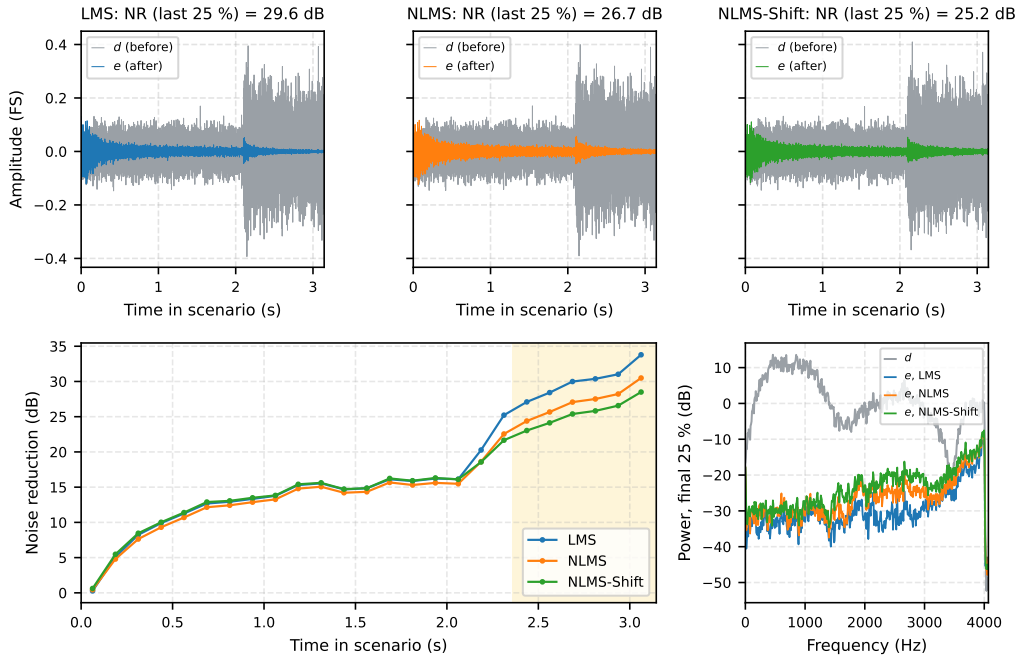


Figure 10: On-board Signal Tap captures of LMS, NLMS and NLMS-Shift on the same amplitude-step noise, from the start of the scenario: primary input d and output e (top), running noise reduction in 0.125 s windows (bottom left; shaded: last 25 % of the common window) and power spectra over that window (bottom right). Every output sample of all three captures equals the integer model's.

9. Discussion and limitations

The comparison supports a narrow conclusion: the divider-free normalisation keeps NLMS's level-independence at about LMS's cycle count, so it is the better choice when the noise level changes and throughput or energy per sample matters, while LMS remains the smallest when the level is steady. It does not show that NLMS-Shift is better than NLMS in cancellation. Section 8.4 explains why: the shift changes only the step size, by a level-dependent factor between 0.5 and 1, so the choice of K and ϵ for the expected input level matters more than the shift itself, and tonal noise near a power-of-two boundary of $P + \epsilon$ sees a step jump that NLMS does not have. On real recordings, the speech at the primary input limits all three algorithms far more than the algorithm choice does (6.4–17.9 dB against at most 0.6 dB between algorithms). The acoustic paths are simulated, also for the

real recordings; the evaluation does not cover acoustic feedback (closed-loop howling), for which reference-based ANC is not the right structure. Power is a switching-activity proxy until gate-level analysis is done. On the board, the output is proven equal to the model up to the DAC word, but the analogue LINE OUT signal was not recorded, and the absolute noise reduction depends on the input level: with the stimulus 3 dB below the simulation level the board reached 14 to 17 dB instead of 19 to 28 dB on the synthetic scenarios, and on stationary and path-switch noise the model reproduces this at that level. A deployment should therefore set the codec input gain so that the reference uses most of the 12-bit range, or retune the step size for the expected level.

10. Conclusion

ULLAP implements LMS, NLMS and NLMS-Shift on one serial 12-bit, 32-tap core on a low-cost Cyclone IV board, compared under equal tuning effort. NLMS-Shift matches NLMS within the 95% confidence interval on stationary and amplitude-step noise while needing 72 instead of 130 busy cycles per sample, and all three remove 11.7 to 14.9 dB of real recorded noise, or 18.5 to 31.9 dB when the speech is removed from the primary input, which shows that the speech, not the noise, limits the result. NLMS-Shift equals NLMS with a level-dependent step between 0.5 and 1 times the nominal one. On the board, 25 Signal Tap captures of real codec data (724,459 samples) are bit-exact with the integer model, as are the freeze behaviour and the DAC word, and on stationary and path-switch noise the measured noise reduction equals the model's at the input level the board received within 0.2 dB.

Declarations

Funding. [TBD: Authors: funding sources, or "No funding was received for this work."]
Competing interests. [TBD: Authors: competing interests, or "The authors declare no competing interests."]
Author contributions. [TBD: Authors: who did what (free text or CRediT).]
Data and code availability. All RTL, testbenches, scripts and result files are in the project repository [TBD: public URL and Zenodo DOI of

the tagged release]. The real-audio study fetches its MS-SNSD files from a pinned commit and checks them by SHA-256.
Use of generative AI. Documented in the experimental set-up (Section 7).

References

[1] L. Lu, K.-L. Yin, R. C. de Lamare, Z. Zheng, Y. Yu, X. Yang, B. Chen, A survey on active noise control in the past decade–Part I: Linear systems, Signal Processing 183 (2021) 108039. doi:10.1016/j.sigpro.2021.108039.

[2] V. Guidotti, G. Paim, L. M. G. Rocha, E. Costa, S. Almeida, S. Bampi, Power-efficient approximate Newton-Raphson integer divider applied to NLMS adaptive filter for high-quality interference cancelling, Circuits, Systems, and Signal Processing 39 (2020) 5729–5757.

[3] C.-Y. Yao, Y.-Z. Huang, The design of the NLMS adaptive filters using the fast-division approximation with CSD encoded divisors, IEEE Transactions on Circuits and Systems II: Express Briefs 71 (4) (2024) 2459–2463.

[4] Y.-Z. Huang, C.-Y. Yao, Enhancing the power efficiency of the NLMS adaptive IIR filter with fast binary reciprocal approximation, Circuits, Systems, and Signal Processing 45 (2026) 7561–7584. doi:10.1007/s00034-026-03536-z.

[5] T. Münzel, M. Kuntic, M. Molitor, M. Sørensen, A. Daiber, Transportation noise pollution as a cardiovascular risk factor: from epidemiological evidence to mechanistic insights, EXCLI Journal 24 (2025) 1713–1739. doi:10.17179/excli2025-9050.

[6] O. Hahad, M. Kuntic, S. Al-Kindi, I. Kuntic, D. Gilan, K. Petrowski, A. Daiber, T. Münzel, Noise and mental health: evidence, mechanisms, and consequences, Journal of Exposure Science and Environmental Epidemiology 35 (1) (2025) 16–23. doi:10.1038/s41370-024-00642-5.

[7] S. Chadha, K. Kamenov, A. Cieza, The world report on hearing, 2021, Bulletin of the World Health Organization 99 (4) (2021) 242–242A. doi:10.2471/BLT.21.285643.

[8] B. Yang, J. Yin, Z. Ye, S. Yang, L. Wang, Development and testing of an active noise control system for urban road traffic noise, Applied Sciences 14 (1) (2024) 175. doi:10.3390/app14010175.

[9] T. Jiang, J. Liu, C. Peng, S. Wang, Laboratory test of a vehicle active noise-control system based on an adaptive step size algorithm, Applied Sciences 13 (1) (2023) 225. doi:10.3390/app13010225.

[10] L. Zhu, X. Zhong, H. Wang, X. Qiu, S. Wu, D. Mao, An objective and subjective study on global active control of engine noise in a car, Journal of Vibration and Control 30 (9–10) (2024) 2311–2321. doi:10.1177/10775463231174730.

[11] W. Zhu, Z. Gu, X. Chen, P. Xie, L. Luo, Z. Bai, A new hybrid adaptive self-loading filter and GRU-net for active noise control in a right-angle bending pipe of an air conditioner, Sensors 25 (20) (2025) 6293. doi:10.3390/s25206293.

[12] Y. Yin, F. Chen, Acoustic feedback cancellation algorithm for hearing aids based on a weighted error adaptive filter, Electronics 12 (7) (2023) 1528. doi:10.3390/electronics12071528.

[13] H. Husstedt, W. Hilgerdenaar, M. Frenz, F. Denk, J. Tchorz, Evaluation of impulse noise reduction in hearing aids with technical measurements and ratings of discomfort, Acta Acustica 7 (2023) 47. doi:10.1051/aacus/2023042.

[14] P. D. Tetteh, R.-M. O. M. Gyening, J. Arthur, J. A. Owusu, B. A. Kponyo, E. Ahene, J. J. Kponyo, Speech enhancement techniques for hearing aids: a systematic review, Universal Access in the Information Society 25 (3) (2026) 75. doi:10.1007/s10209-026-01342-7.

[15] L. Lu, K.-L. Yin, R. C. de Lamare, Z. Zheng, Y. Yu, X. Yang, B. Chen, A survey on active noise control in the past decade–Part II: Nonlinear systems, Signal Processing 181 (2021) 107929. doi:10.1016/j.sigpro.2020.107929.

[16] F. B. Félix, M. de Castro Magalhães, G. de Souza Papini, Improved active noise control algorithm based on the convex combination method, Journal of the Brazilian Society of Mechanical Sciences and Engineering 43 (2021) 163. doi:10.1007/s40430-021-02866-0.

[17] F. Li, H. Ding, Z. Wu, F. Qian, T. Liu, A. Iqbal, An improved stability compensation for feedforward active noise control systems with acoustic feedback, Journal of Mechanical Science and Technology 38 (2024) 507–518. doi:10.1007/s12206-024-0101-5.

[18] H. H. Thannoon, I. A. Hashim, Hardware implementation of a high-speed adaptive filter using a combination of systolic and convex architectures, Circuits, Systems, and Signal Processing 43 (2024) 1773–1791. doi:10.1007/s00034-023-02539-4.

[19] M. Jagadeeswari, P. Rishabavarthani, S. Kumarganesh, S. Kannadhasan, Enhanced adaptive filtering in audio systems: normalized least mean square with cut-set retiming, Analog Integrated Circuits and Signal Processing 126 (1) (2026) 12. doi:10.1007/s10470-025-02537-3.

[20] E. Karakaya, O. Aytar, FPGA implementation of NLMS and VSLMS algorithms for underwater acoustics applications, IEEE Access 14 (2026) 53860–53872. doi:10.1109/ACCESS.2026.3681282.

[21] S. Park, D. Park, Low-power FPGA realization of lightweight active noise cancellation with CNN noise classification, Electronics 12 (11) (2023) 2511. doi:10.3390/electronics12112511.

[22] F. Abdi, W. Cichalewski, W. Jałmużna, Ł. Butkowski, J. Branlard, A. Bellandi, G. Jabłoński, FPGA-based implementation of an adaptive noise controller for continuous wave superconducting cavity, Electronics 13 (1) (2024) 155. doi:10.3390/electronics13010155.

[23] M. Nagabushanam, S. Chakrasali, S. L. Gangadharaiah, S. H. Patel, G. Ramaiah, R. Basak, An optimized VLSI implementation of the least mean square (LMS) adaptive filter architecture on the basis of distributed arithmetic approach, Journal of The Institution of Engineers (India): Series B 106 (3) (2025) 861–870. doi:10.1007/s40031-024-01028-9.

[24] M. M. A. da Rosa, P. Ü. da Costa, E. A. C. da Costa, S. J. M. Almeida, G. Paim, S. Bampi, Design of a low power and robust VLSI power line interference canceler with optimized arithmetic operators, Analog Integrated Circuits and Signal Processing 112 (2) (2022) 247–261. doi:10.1007/s10470-022-02050-x.

[25] L. F. d. S. Coelho, L. Lovisolo, M. P. Tcheou, Adaptive filtering with reduced computational complexity using SOPOT arithmetic, IEEE Transactions on Circuits and Systems I: Regular Papers 69 (2) (2022) 746–756.